Availability

System that can handle millions of requests even if it crashes when a single server fails?

One important distinction before we begin: **availability** is not the same as **reliability**. A system can be highly available (always up) but unreliable (sometimes gives wrong answers).

Measuring Availability

Availability is typically expressed as a percentage of uptime over a given period. The formula is straightforward:

```
Availability = Uptime / (Uptime + Downtime)
```

For example, if a system was up for 364 days and down for 1 day in a year:

Availability = 364 / 365 = 99.73%

That single day of downtime drops you below "three nines" availability. When you frame it as "one day," it sounds acceptable.

The "Nines" of Availability

Availability is often described in terms of "nines." Each additional nine dramatically reduces allowed downtime:

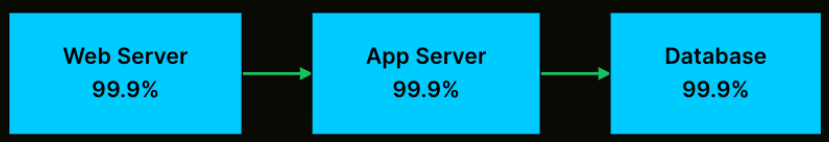
Availability	Downtime per Year	Downtime per Month	Downtime per Week
99% (two nines)	3.65 days	7.3 hours	1.68 hours
99.9% (three nines)	8.76 hours	43.8 minutes	10.1 minutes
99.99% (four nines)	52.6 minutes	4.38 minutes	1.01 minutes
99.999% (five nines)	5.26 minutes	26.3 seconds	6.05 seconds
99.9999% (six nines)	31.5 seconds	2.63 seconds	0.6 seconds

Availability in Series vs Parallel

How you combine components dramatically affects overall availability.

Components in Series

When components are in series, meaning all must work for the system to function, availability multiplies:

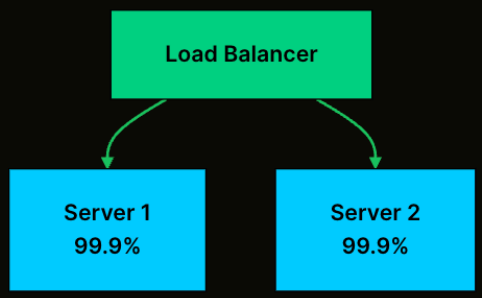


Overall = 99.9% × 99.9% × 99.9% = 99.7%

Each component in the chain reduces overall availability. You started with three components, each at "three nines," but the combined system is below three nines. Add more components in series, and availability keeps dropping.

Components in Parallel

When components are in parallel, meaning any can handle the request, availability improves dramatically:



For both servers to be down simultaneously, both must fail at the same time:

Failure probability = 0.1% × 0.1% = 0.0001%

Availability = 100% - 0.0001% = 99.9999%

Two servers with 99.9% availability each give you nearly six nines when running in parallel. This is the power of redundancy.

Common Failure Modes

To design for availability, you must understand how things fail. Failures do not ask permission, and they rarely happen at convenient times. Knowing the common failure modes helps you prepare for them.

Hardware Failures

Everything physical eventually breaks. The question is when, not if.

Component	Typical Failure Rate	MTBF
Hard Drive (HDD)	2-4% per year	300,000 hours
SSD	0.5-1% per year	1-2 million hours
Server	2-4% per year	300,000 hours
Network Switch	1-2% per year	500,000 hours
Power Supply	1-3% per year	400,000 hours

MTBF = Mean Time Between Failures.

At scale, hardware failures are not exceptional events. They are routine. A data center with 10,000 servers will see hundreds of hardware failures per year. If your architecture cannot handle a server dying at any moment, you do not have a highly available system.

Software Failures

Hardware breaks randomly. Software breaks creatively.

- **Bugs:** Code defects that cause crashes or incorrect behavior
- **Memory leaks:** Gradual resource exhaustion
- **Deadlocks:** Processes waiting on each other indefinitely
- **Cascading failures:** One failure triggering failures in dependent systems

Software Failures: Memory Leaks

(Gradual resource exhaustion)

- **What it is:** When a software program runs, it requests memory (RAM) from the operating system to store temporary data. A memory leak occurs when the program forgets to "give back" or free up that memory once it's done using it.

- **The Impact:** Because the leaked memory is never released, the available free memory on the server slowly dwindles over time. The application will gradually become slower and slower as it struggles to find free memory, until it eventually crashes entirely (an "Out of Memory" or OOM error).

Network Failures

Networks fail in ways that are subtle, intermittent, and painful to debug.

- **Packet loss:** Data does not reach its destination
- **Latency spikes:** Delays in communication
- **Partition:** Network split isolates groups of servers
- **DNS failures:** Name resolution stops working

Human Errors

Here's the uncomfortable reality: many studies attribute **70–80% of outages** to human error, not hardware, not software, but people.

Common examples:

- **Configuration mistakes:** wrong environment variable, typo in a config file
- **Failed deployments:** bad code or broken migrations pushed to production
- **Accidental deletions:** running the wrong command in the wrong place
- **Capacity planning errors:** underestimating traffic for a launch

Network Failures: Network Partition

(Network split isolates groups of servers)

- **What it is:** In modern infrastructure, applications are usually hosted across many different servers that constantly talk to each other to share data and balance the workload. A network partition happens when a network link fails (e.g., a router dies, or a cable is cut), severing the communication line between two or more groups of these servers.
- **The Analogy:** Imagine a city split by a river with a single bridge. If that bridge collapses, the city is partitioned into the East side and the West side. Both sides still have power, running water, and functioning businesses, but they cannot communicate or trade with each other.

- **The Impact:** The servers are technically still "alive," but they are blind to each other. This often leads to a dangerous scenario called **"Split-Brain,"** where both isolated groups of servers think the *other* group has died. If both groups try to take control of the database and write conflicting data at the same time, it can cause severe data corruption.

Network Failures: DNS Failures

(Name resolution stops working)

- **What it is:** Computers communicate using IP addresses (like `192.168.1.1` or `142.250.190.46`), but humans use domain names (like `google.com` or `amazon.com`). The **Domain Name System (DNS)** is the system that translates the human-friendly name into the computer-friendly IP address. A DNS failure means this translation service has broken down.
- **The Impact:** When DNS fails, users will see "Site Not Found" errors in their browser. The frustrating part of a DNS failure is that your actual web servers, databases, and network might be running perfectly, but nobody can reach them because they don't know the "address" to send the traffic to.

Redundancy: The Foundation of Availability

If there is one concept that underpins all of availability, it is redundancy. The logic is simple: if you have only one of something, when it fails, you have zero. If you have two, when one fails, you still have one.

Redundancy means having backup components that can take over when primary components fail.

Active-Passive (Standby)

In an active-passive configuration, one component handles all the work while another waits idle as a backup. When the active component fails, the passive one takes over.

Active-passive mode is commonly used in situations where you want a single source of truth and controlled writes like databases, stateful services, and systems requiring a single leader.

Pros

- Simple to reason about
- Standby typically uses fewer resources
- Clear source of truth

Cons

- Failover takes time (detection + promotion + routing changes)
- Standby may not be truly “production-ready” because it isn’t tested under real load
- Potential for split-brain problem

The standby can be configured in different states of readiness:

Standby Type	State	Failover Time	Cost
Cold Standby	Powered off, needs to boot	Minutes	Lowest
Warm Standby	Running but not receiving traffic	Seconds to minutes	Medium
Hot Standby	Running, data synchronized, ready to serve	Seconds	Highest

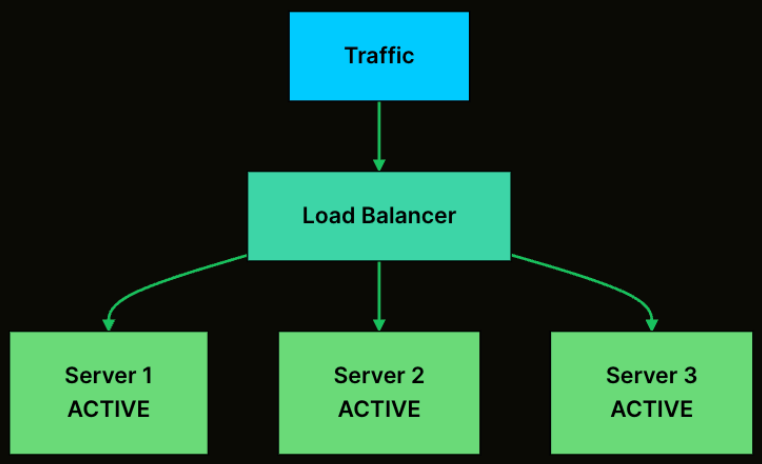
Cold standby is cheapest but slowest. The backup server is not running, so failover requires booting the machine, starting services, and potentially restoring data. This might take 5-15 minutes, which is too slow for most production systems but acceptable for disaster recovery.

Warm standby keeps the backup running and configured, but not actively processing requests. It might be receiving replicated data but is not in the load balancer pool. Failover involves adding it to the pool and possibly promoting it, which takes seconds to a few minutes.

Hot standby is the most expensive but fastest. The backup is fully synchronized and ready to serve immediately. For databases, this often means synchronous replication where every write is confirmed on both primary and standby before acknowledging the client.

Active-Active

In an active-active configuration, all components handle traffic simultaneously. There is no distinction between primary and backup because every node is doing real work.



When one node fails, the load balancer simply stops sending traffic to it. There is no failover process because the other nodes were already handling traffic. The remaining nodes absorb the additional load.

Pros

- No failover delay
- All nodes tested under real load
- Better resource utilization

Cons

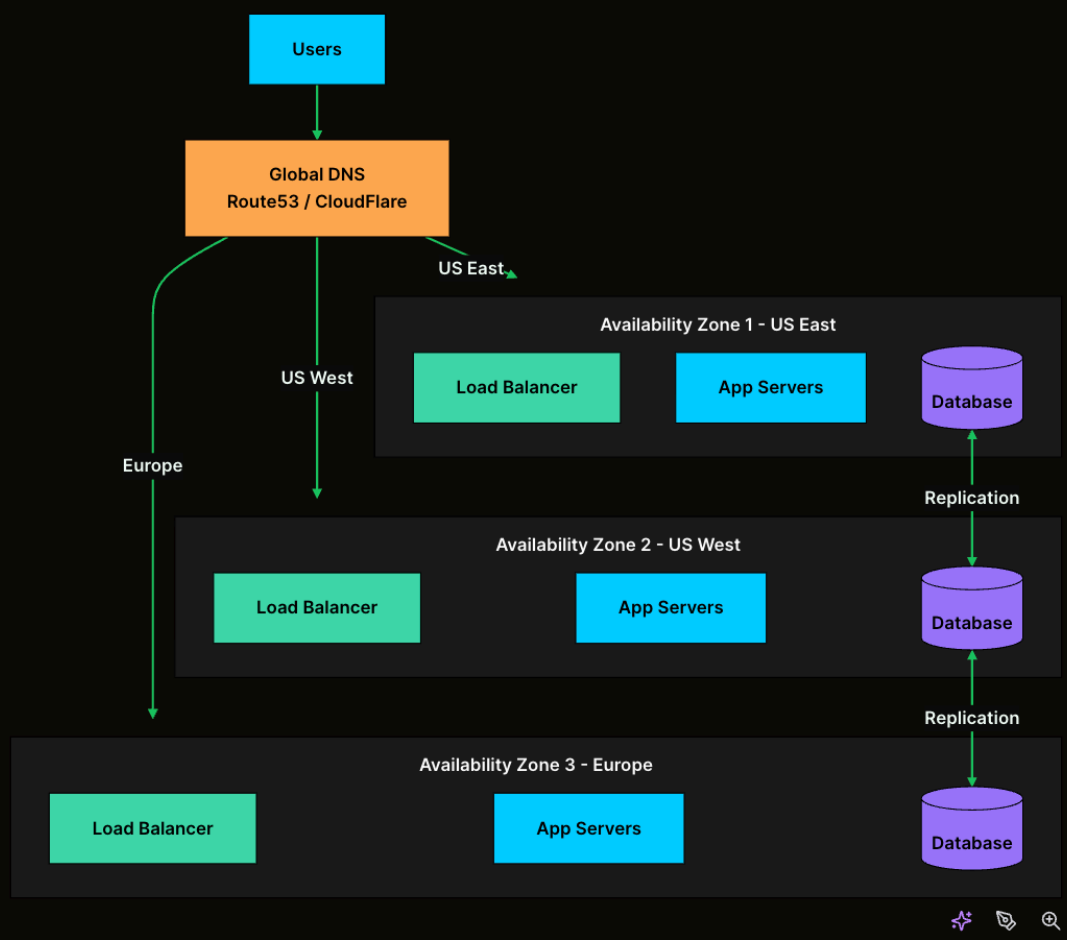
- More complex
- Must handle data consistency across nodes
- equires stateless design or shared state

The key requirement for active-active is that requests can be handled by any node. This works naturally for stateless services where each request is independent. For stateful services, you need either shared storage (like a database or Redis) or sticky sessions (which reduces availability benefits).

Geographic Redundancy

Redundancy within a single data center protects against hardware failures, but what if the entire data center goes offline? Power outages, network cuts, natural disasters, or even a backhoe cutting a fiber line can take down an entire facility.

Geographic redundancy distributes your system across multiple physical locations:



Cloud providers offer different levels of geographic redundancy:

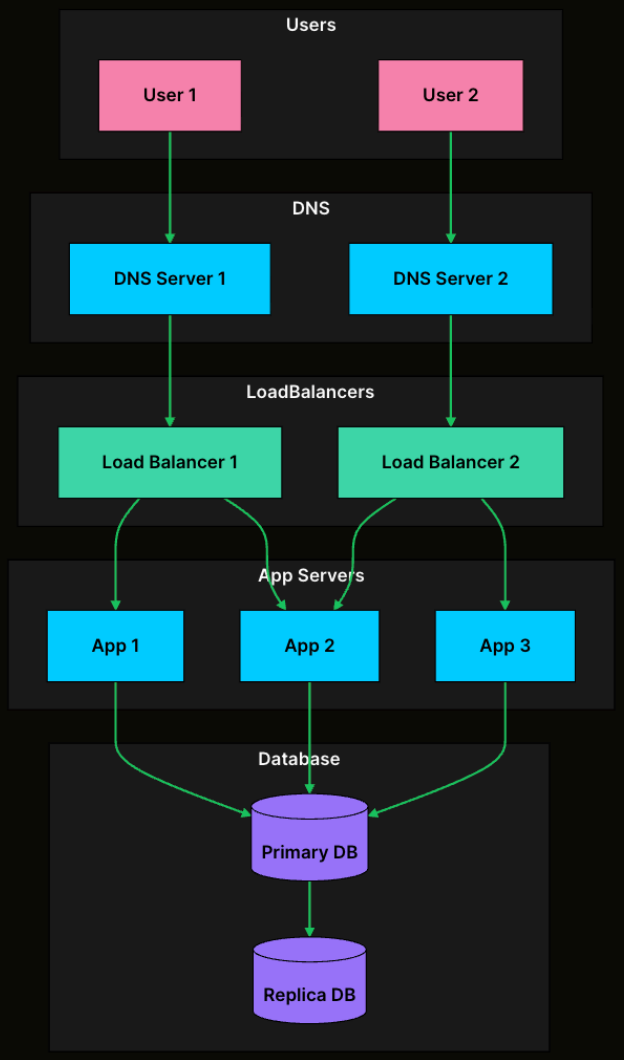
Level	What It Is	Protects Against	Latency Impact
Availability Zones (AZs)	Separate data centers in the same region, connected by low-latency links	Single data center failure	Minimal (1-2ms)
Regions	Geographically separate areas (e.g., US-East vs US-West)	Regional disasters, wide-spread outages	Significant (50-100ms+)
Multi-Cloud	Different cloud providers (AWS + GCP)	Cloud provider outages	Variable

Availability Zones are the sweet spot for most applications. They provide meaningful isolation (separate power, cooling, and network) while keeping latency low enough for synchronous replication. Most cloud-native applications deploy across at least two AZs.

Multi-region deployment is necessary for global applications or those requiring disaster recovery from regional events. The challenge is data replication, since synchronous replication across regions adds significant latency. Most multi-region systems use asynchronous replication and accept some data loss in a disaster (typically seconds to minutes of transactions)

Redundancy Across Layers

A chain is only as strong as its weakest link. If you have redundant app servers but a single database, the database is your single point of failure. True high availability requires redundancy at every layer of your stack.



Notice that redundancy gets harder as you move down the stack. Adding more web servers is trivial. Adding database replicas with automatic failover requires careful engineering.

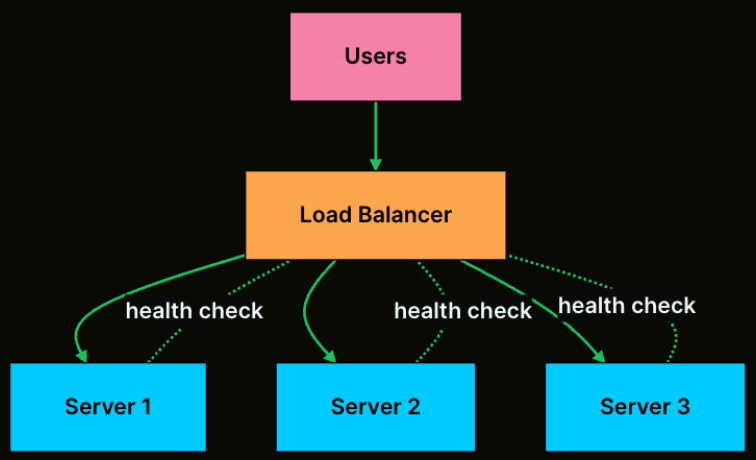
> **NOTE**
>
> Redundancy is not free. Every backup server, every replica, every additional availability zone costs money. The question is whether that cost is justified by the reduction in down-

High Availability Patterns

Patterns are reusable solutions to common problems. The following patterns appear repeatedly in highly available systems.

Pattern 1: Load Balancer with Multiple Backends

The most common and fundamental pattern for stateless services. A load balancer distributes traffic across multiple servers, automatically routing around failures.

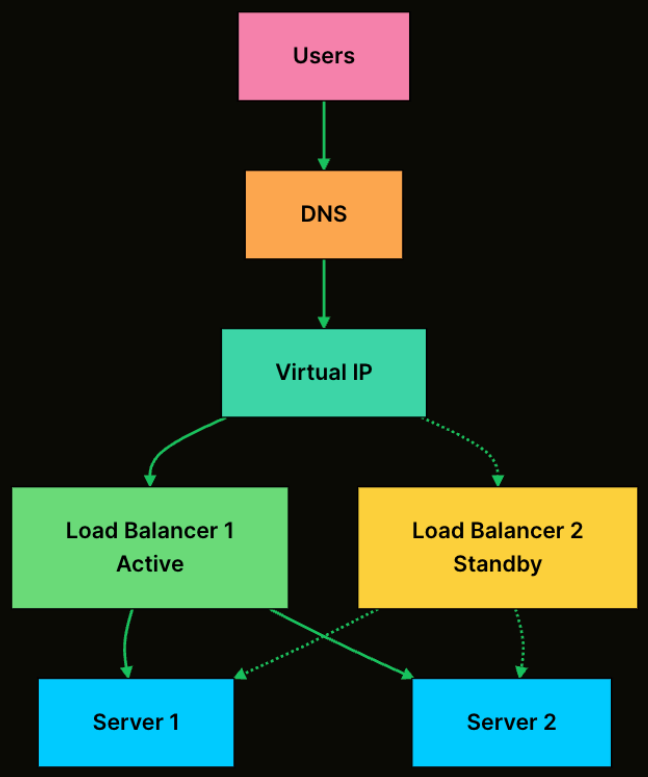


How it provides high availability:

- Load balancer continuously monitors backend health
- Failed servers are automatically removed from rotation
- Traffic redistributes to healthy servers within seconds
- New servers can be added without any downtime

Load Balancer Redundancy

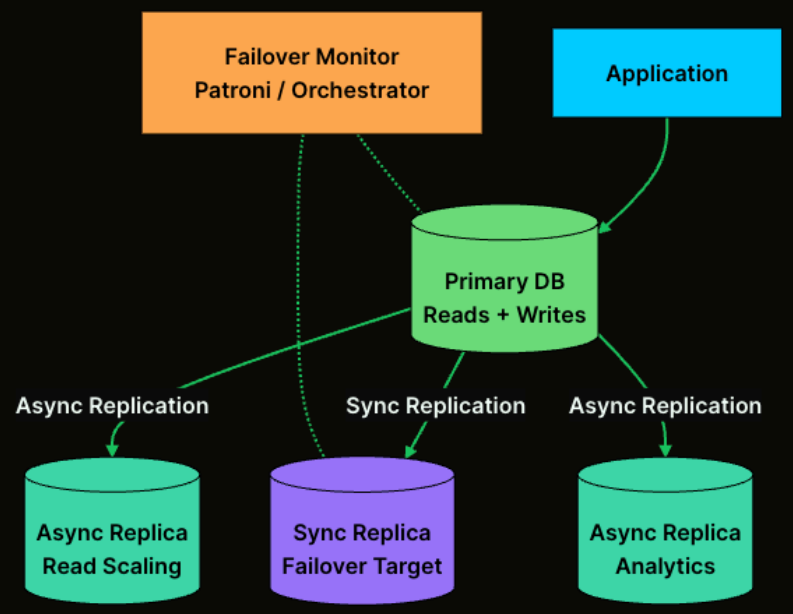
The load balancer itself is a single point of failure. For true high availability, you need redundant load balancers:



Cloud providers handle this automatically. AWS ALB, Google Cloud Load Balancer, and Azure Load Balancer are all managed services with built-in redundancy. On-premises, you might use keepalived with a virtual IP that floats between two HAProxy instances.

Pattern 2: Database Replication with Automatic Failover

Databases are stateful and cannot simply be load-balanced like web servers. Database high availability requires replication and careful failover management.



Synchronous vs Asynchronous Replication

Type	How It Works	Data Loss	Performance Impact
Synchronous	Write confirmed only after replica acknowledges	Zero (RPO = 0)	Higher latency (wait for replica)
Asynchronous	Write confirmed immediately, replica catches up later	Possible (seconds to minutes)	No impact on write latency
Semi-synchronous	Wait for at least one replica, others async	Minimal	Moderate impact

Synchronous replication guarantees zero data loss but adds latency. Every write must wait for the replica to confirm. If your replica is in a different region, this adds significant latency (50-100ms per write).

Asynchronous replication has no performance impact but can lose data. If the primary fails, any writes not yet replicated are lost. The "replication lag" is typically seconds but can grow during high load.

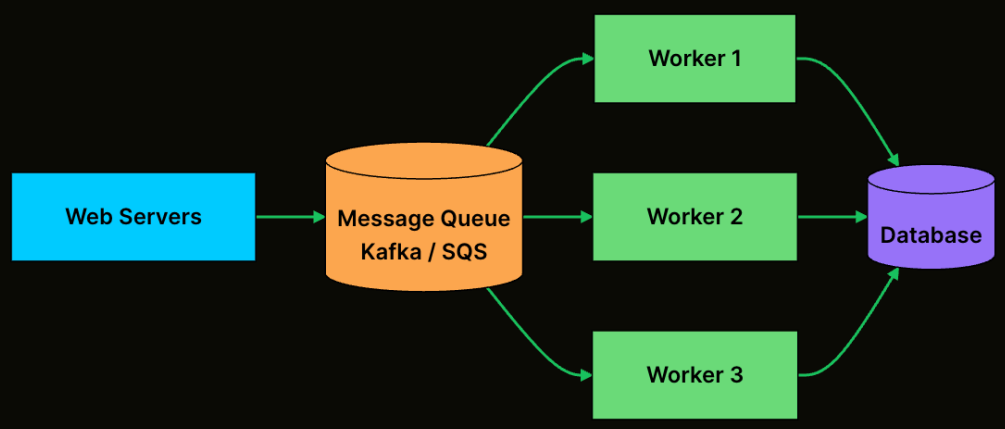
Most production systems use synchronous replication for the failover target and asynchronous replication for read replicas and analytics.

Why Use Both in One System?

- **The Safety Net:** It uses **one Synchronous Replica** purely as an emergency backup. It eats the latency cost on this one connection to ensure zero data loss (RPO = 0).
- **The Workhorses:** It uses **Asynchronous Replicas** to handle the heavy lifting. Instead of funneling every single user read and analytics report through the Primary DB (which would cause it to crash under heavy load), it routes all that "read-only" traffic to the green Async Replicas. Because these are just handling reads, it doesn't matter if they are a few milliseconds behind the primary ("replication lag").

Pattern 3: Queue-Based Load Leveling

When downstream services cannot handle peak load, use a queue to buffer requests and process them at a sustainable rate.



How it provides high availability:

- Decouples producers from consumers
- Buffers traffic spikes that would overwhelm the database
- Workers can fail and restart without losing messages
- Can scale workers independently based on queue depth

This pattern is essential for handling bursty traffic. A flash sale might generate 100x normal traffic for a few minutes. Without a queue, the database would be overwhelmed. With a queue, orders accumulate and are processed at a sustainable rate.

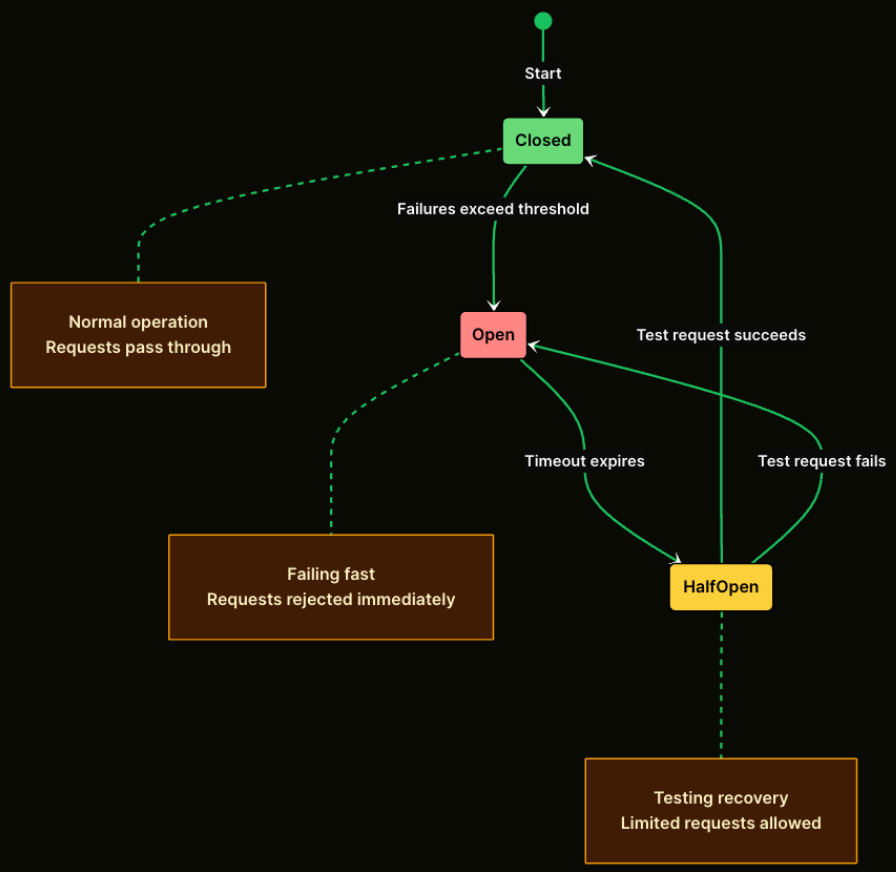
If database replication is about protecting the data *after* it's saved, a message queue is about protecting the database from getting crushed *before* the data is saved.

Instead of the Web Servers talking directly to the Database, they drop the orders into a Message Queue (like Kafka, AWS SQS, or RabbitMQ).

1. **Decoupling:** The Web Servers just drop the message in the box and instantly tell the user, "Order received!" They don't wait for the database. This keeps the web servers incredibly fast and responsive.
2. **Buffering:** The Queue acts as a massive waiting room. It doesn't care if it receives 10 messages or 10,000 messages a second; it just holds them in a neat line.
3. **Pacing (Load Leveling):** The "Workers" (small background programs) pull messages off the queue *at their own pace*. If the database can only safely handle 50 writes per second, you set up enough workers to process exactly 50 messages per second.

Pattern 4: Circuit Breaker

When a dependency fails, continuing to call it wastes resources and can cause cascading failures. The circuit breaker pattern prevents this by failing fast.



Circuit Breaker States

State	Behavior	Transitions
Closed	Normal operation. All requests pass through. Track failure rate.	→ Open: when failure rate exceeds threshold
Open	Fail fast. All requests immediately rejected with error.	→ Half-Open: after timeout period
Half-Open	Testing. Allow limited requests through.	→ Closed: if test succeeds / → Open: if test fails

The Problem: Cascading Failures

Imagine your main Web App relies on a separate Payment Service to process checkouts.

- Suddenly, the Payment Service goes down or gets extremely slow.
- Your Web App keeps sending requests to the Payment Service, waiting 30 seconds for each one before giving up.
- Because the Web App is stuck waiting, its own memory and processing threads get completely tied up.
- Soon, your Web App crashes too. And then the mobile app that talks to the Web App crashes. One broken service takes down the entire company.

The Solution: The Circuit Breaker

Instead of the Web App talking directly to the Payment Service, it routes all traffic through a "Circuit Breaker" mechanism in the code. This mechanism monitors the health of the connection and operates in one of three states:

1. CLOSED (Normal Operation)

- *How it works:* The Web App sends a request, the breaker lets it pass through to the Payment Service, and the response comes back. Life is good. But the breaker is secretly counting failures.

2. OPEN (Failing Fast)

- *How it works:* If the breaker sees too many failures in a row (e.g., 5 errors in 10 seconds), it trips to the **Open** state. Now, when the Web App tries to call the Payment Service, the breaker *instantly* blocks it and returns an error.
- *The Benefit:* This is called **"Failing Fast."** The Web App doesn't waste 30 seconds waiting for a dead service. It gets an instant rejection, allowing it to quickly show the user an error message ("Payments currently unavailable") while the rest of the app stays fast and healthy. It also gives the Payment Service time to recover without being hammered by traffic.

3. HALF-OPEN (Testing the Waters)

- *How it works:* After a set timeout (e.g., 60 seconds of being Open), the breaker cautiously transitions to **Half-Open**. It allows exactly *one* test request to pass through to the Payment Service.

- If that test request **succeeds**, the breaker assumes the Payment Service is fixed. It snaps back to **CLOSED**, and traffic flows normally again.
- If that test request **fails**, the breaker knows the service is still broken. It immediately snaps back to **OPEN** and starts the 60-second timeout clock all over again.